SmartPot

DOCUMENTACIÓN DEL COMPONENTE

SmartPot-AI

El asistente inteligente de SmartPot

Sistema experto, lógica difusa, modelos de aprendizaje automático, agente reactivo, pronóstico, análisis de flota y aprendizaje continuo con las lecturas de los cultivos reales.



DOCUMENTO
SmartPot-AI

VERSIÓN
1.2 · octubre 2026

EQUIPO
SmartPotTech

PROYECTO
smartpot.app

DOCUMENTO

Ficha del documento

CAMPO	VALOR
Proyecto	SmartPot · smartpot.app
Componente	SmartPot-AI
Versión	1.2 · octubre 2026
Alcance	Cómo razona el asistente, aprendizaje continuo, análisis de flota, contrato, configuración, pruebas y operación
Documentación de la plataforma	Documentación técnica, recorrido del proyecto, ciclo de vida y diagramas generales
Mantenimiento	Se genera desde `docs/` de este repositorio con las herramientas de `.github/docs/tools`; se actualiza con cada cambio del componente

CONTENIDO

Contenido

1. Propósito

2. Arquitectura del componente

3. Cómo razona una evaluación

4. Aprendizaje continuo

5. Análisis de flota

6. Contrato

7. Configuración

8. Pruebas

9. Operación

PARTE I

El componente

SECCIÓN 01

Propósito

En palabras simples

SmartPot-AI es el agrónomo de la plataforma. La API le manda la última lectura de un cultivo, su historial, sus actuadores, dónde está y el clima de afuera; el asistente responde cómo está (un índice de salud de 0 a 100), por qué, qué va a pasar en las próximas horas y qué conviene hacer. Además aprende sin parar de las lecturas de los cultivos reales de cada especie, sin saber de qué cuenta viene cada una. Es un servicio interno: solo la API lo consulta, con un token.

TÉCNICA	QUÉ RESUELVE
Sistema experto	Diagnóstico por variable y conclusiones encadenadas con su explicación
Lógica difusa	Índice de salud que tolera la incertidumbre de los sensores
Aprendizaje automático	Ventilación, corrección de pH y lecturas atípicas
Pronóstico	Tendencia de cada variable y horas hasta salir del rango
Agente reactivo	Acciones sobre los actuadores que el cultivo sí tiene
Análisis de flota	Ranking, problemas del entorno, grupos y acciones en bloque
Consejo de lugar	Si el lugar le sirve a la especie y cuándo conviene moverla
Aprendizaje continuo	Modelos que mejoran con las lecturas reales y solo reemplazan al vigente si lo superan
Aceleración C++ nativa	Cómputo numérico directo en C++20 (Theil-Sen $O(M)$ lineal, Sugeno SIMD, calidad IQR y normalización) con C ABI y fallback

SECCIÓN 02

Arquitectura del componente

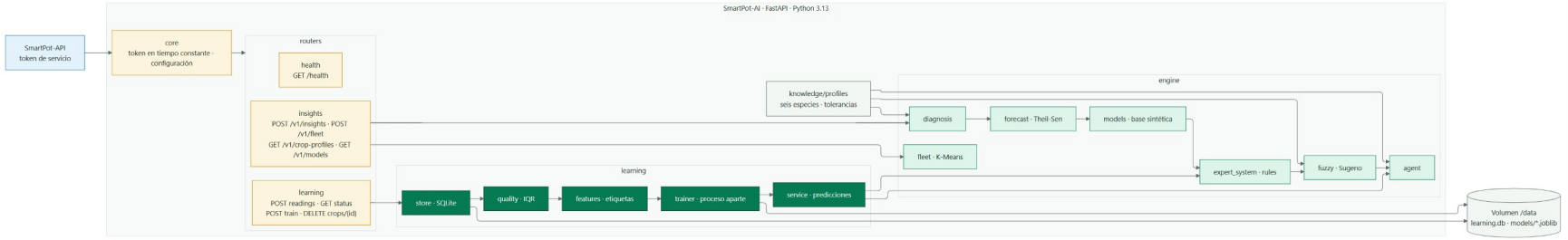


Figura 1 · *SmartPot-AI por dentro*

Motor Nativo C++20 de Aceleración Numérica

Para optimizar de forma extrema la latencia, el uso de memoria RAM y la escalabilidad concurrente bajo ráfagas intensivas de peticiones, los algoritmos numéricos de cálculo pesado han sido reescritos directamente en **C++20** (`src/native/`):

- **Estimador Theil-Sen $O(M)$ Lineal**: El cálculo tradicional de Theil-Sen requiere comparar todos los pares de lecturas temporales ($M = N(N-1)/2$). En Python/NumPy esto creaba matrices temporales masivas (`np.triu_indices`) y ordenaciones $O(M \log M)$. En C++20, se utiliza selección lineal en tiempo $O(M)$ con `std::nth_element`, logrando entre **3.6x y 50x de aceleración** con huella de memoria nula y cero fragmentación del heap.
- **Inferencia Difusa Sugeno Vectorizada**: Funciones de pertenencia trapezoidal branchless (sin saltos condicionales) y ponderación de reglas mediante constantes en tiempo de compilación (`constexpr`), con cálculo en tiempo sub-milisegundo.
- **Control de Calidad de Datos (IQR)**: Evaluación de completitud, validación física contra límites de sensores y detección de espigas anómalas en lotes de hasta 20 000 lecturas mediante percentiles en tiempo lineal.
- **Transformación de Características y Codificación Circular**: Normalización de tolerancia relativa y codificación trigonométrica continua de hora solar ($\sin$, $\cos$).
- **C ABI y Fallback Transparente**: Enlazado mediante `ctypes` (`app/engine/native_bridge.py`) con estructuras de memoria empaquetadas a 8 bytes (`#pragma pack(push, 8)`). Si la biblioteca no está compilada en el entorno, el sistema degrada de manera transparente a las rutinas estándar de Python garantizando 100% de disponibilidad.

PARTE II

Razonamiento

SECCIÓN 03

Cómo razona una evaluación

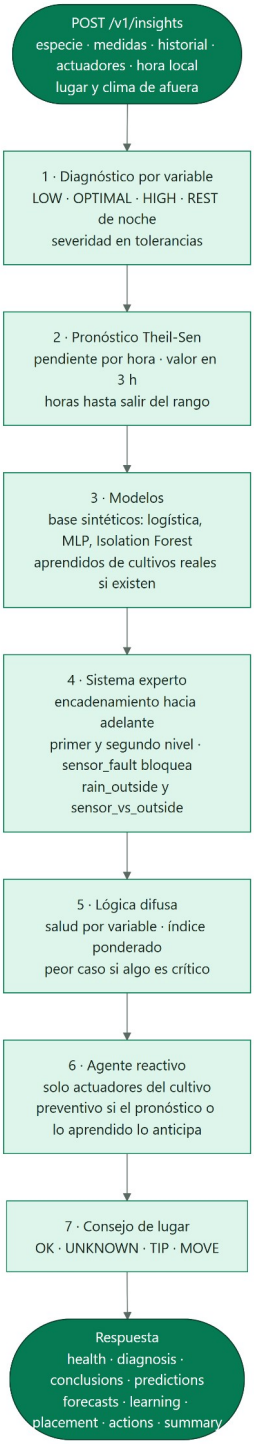


Figura 2 · *De la lectura a la respuesta*

PASO	DETALLE
Base de conocimiento	Rangos ideales de lechuga, tomate, fresa, albahaca, espinaca y pimentón en la escala de los sensores; una variable fuera de rango se mide en tolerancias y desde una completa es crítica
Noche	Entre las 22:00 y las 6:00 la luz baja es descanso (`REST`) y no resta salud
Modelos base	Se entrenan al arrancar con 4000 muestras sintéticas y semilla fija; `GET /v1/models` informa su exactitud
Reglas	Encadenamiento hacia adelante con prioridades; `sensor_fault` bloquea las acciones si la lectura es imposible
Agente	Riego (no con lluvia sobre un cultivo al aire libre), luz ultravioleta de día (y apagarla de noche), ventilación, humidificador y dosificadores, solo si el cultivo tiene el actuador; la API aplica el enfriamiento y el modo automático
Lugar y clima	Con `placement` y `weather`: consejo `OK`, `UNKNOWN`, `TIP` o `MOVE` según la luz que pide la especie; reglas `rain_outside` (0,5 mm o más) y `sensor_vs_outside` (8 °C o más de diferencia con el clima)

SECCIÓN 04

Aprendizaje continuo

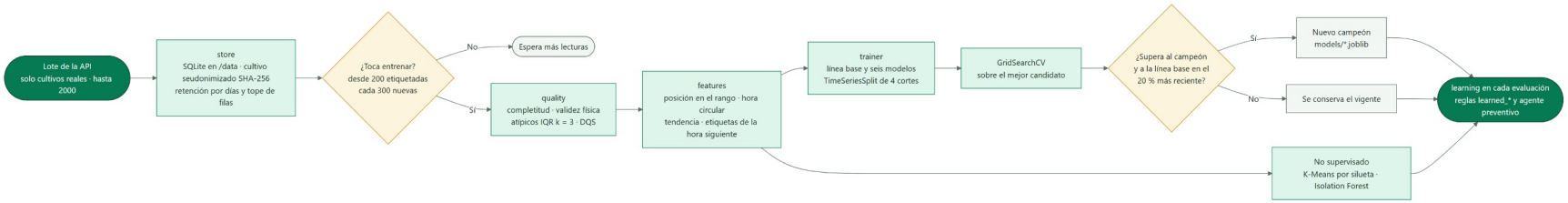


Figura 3 · *Ciclo del aprendizaje continuo*

Solo aprende de cultivos **reales**: la API no envía las lecturas de los cultivos virtuales, que son sintéticas. Cada especie entrena por separado y en un proceso aparte para no frenar las evaluaciones; una tarea queda pendiente, con su razón, mientras falten datos o casos. Borrar un cultivo borra sus lecturas.

SECCIÓN 05

Análisis de flota

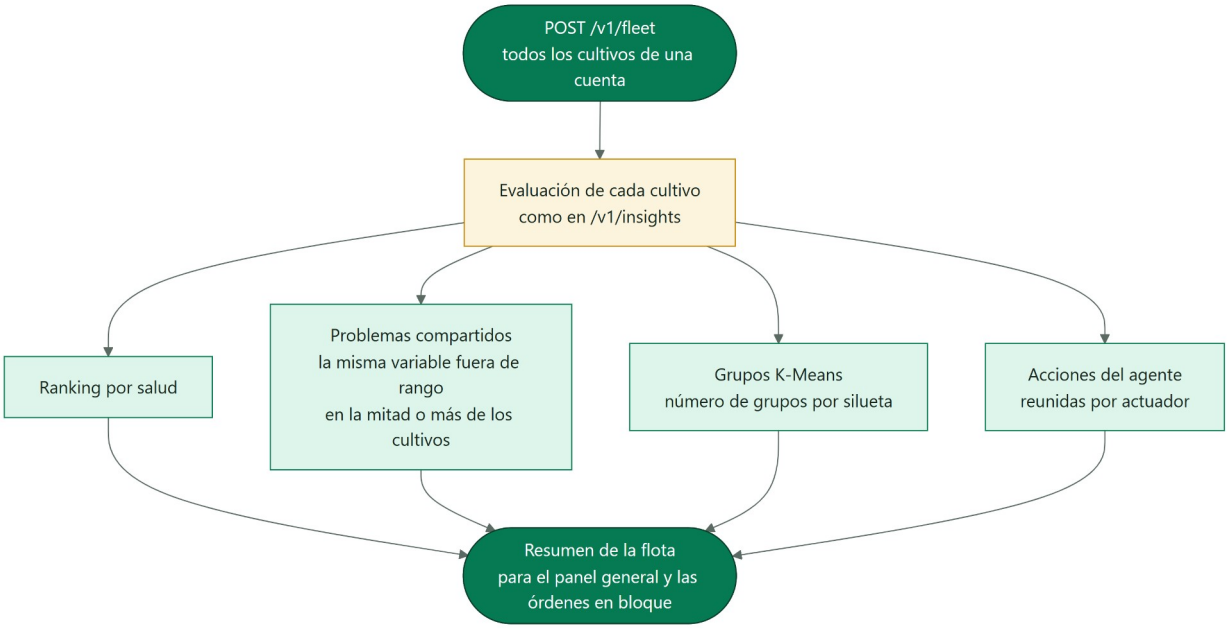


Figura 4 · *Análisis de todos los cultivos de una cuenta*

PARTE III

Operación

SECCIÓN 06

Contrato

Todas las rutas `/v1` exigen `Authorization: Bearer <SMARTPOT_AI_TOKEN>`; el README trae ejemplos de petición y respuesta.

MÉTODO	RUTA	DESCRIPCIÓN
GET	`/health`	Estado de los modelos y del almacén de aprendizaje
POST	`/v1/insights`	Evalúa una lectura
POST	`/v1/fleet`	Analiza todos los cultivos de una cuenta
GET	`/v1/crop-profiles`, `/v1/models`	Base de conocimiento y exactitud de los modelos base
GET	`/v1/engine`	Telemetría, versión y compilación del motor C++20
POST, GET	`/v1/learning/readings`, `/v1/learning/status`	Recibe lecturas reales por lote; informa lo aprendido por especie
POST, DELETE	`/v1/learning/train`, `/v1/learning/crops/{cropId}`	Entrena ya; olvida un cultivo borrado

SECCIÓN 07

Configuración

VARIABLE	USO
`SMARTPOT_AI_TOKEN`	Token de servicio, al menos 24 caracteres
`DOCS_ENABLED`, `LOG_LEVEL`	Documentación interactiva y nivel de registro
`MODEL_SEED`	Semilla de los modelos base
`DATA_DIR`	Carpeta de SQLite y modelos (volumen `/data` en la imagen)
`LEARNING_MIN_SAMPLES`, `LEARNING_RETRAIN_EVERY`	Lecturas para el primer entrenamiento y cada cuántas nuevas se reentrena

VARIABLE	USO
`LEARNING_CHECK_SECONDS`, `LEARNING_RETENTION_DAYS`	Cada cuánto revisa si toca entrenar y cuántos días guarda

SECCIÓN 08

Pruebas

`uv run ruff check .` y `uv run pytest`: 92 pruebas sobre la base de conocimiento, paridad matemática y numérica exacta entre Python y C++ ($< 10^{-6}$), encadenamiento de reglas, la monotonía del índice difuso, la exactitud mínima de los modelos, el pronóstico, el agente, la lluvia y el sensor frente al clima, el consejo de lugar, la flota, el contrato HTTP, telemetría del motor (`/v1/engine`) y el aprendizaje continuo (seudonimización, poda, calidad, etiquetas, modelos que superan a la línea base, campeón y retador, persistencia).

SECCIÓN 09

Operación

TAREA	CÓMO
Imagen	`ghcr.io/smartpottech/smartpot-ai`: dependencias con uv, usuario `1000`, solo lectura; `/data` en un volumen para conservar lo aprendido
Red	Solo en la red interna, sin salida a internet ni puertos publicados
Entrenamientos	`docker logs smartpot-ai` muestra cada uno; la PWA los resume en Aprendizaje
Despliegue	Cada cambio en `main` pasa por el CI, publica la imagen y pide el despliegue central de `.github`